Lab 1: Introduction to Rosbots, ROS2 packages, and remote control

The goal of this lab is to get you oriented to the bots we will be using this semester. I'll be referring to them as `Rosbots`, but they're actually JetRovers from the company HiWonder. These bots are equipped with the following components:

Component	Description	Purpose
Jetson Orin Nano	mini-PC	Brains and interface of the robot
STM-32	micro-controller	Controls servos, drive motors, and interfaces with IMU and joystick
Ackermann Drive	Chassis	The chassis of the robot uses an Ackermann Drive drivetrain. This means that it turns by turning the front wheels, just like a car. This will make navigation trickier, but we'll introduce tools to help us!
GPLidar A1	Scanning LiDAR	Detects distances to obstacles in a plane around the front of the robot
Orbbec DaBai DCW2	Depth Camera	Color camera + depth information (RGBD), located at the end of the gripper.
5R Robot Arm	Servos	Robot arm with 5 rotary joints and a gripper, controlled by 6 servo motors in a serial bus. We'll control this in a future lab.

Connection Information

There are 3 options to connect to the Rosbots. In all approaches, the Rosbot will have the username `ubuntu` and password `ubuntu`

1. **Remote Desktop**: Connect your WiFi to the Rosbot's access point, using the password `hiwonder`

. The specific Access Point SSID for your bot is located on the OLED screen at the front of the robot. Then, download and install the software Nomachine (Download Links). From Nomachine, connect to your bot using the username and password above. **NOTE**: Only one device can be connected through Nomachine at a time.

2. **SSH over WiFi**: Once you have connected to the Rosbot's access point, as described in the Remote Desktop description, open a terminal and enter the command `ssh ubuntu@192.168.149.1`. You'll be prompted for the password (`ubuntu`).
3. **SSH over wire**: Depending on your computer's firewall settings, you may be able to connect directly to the bot with a wire. We have USB-A to USB-C cables available in the lab. To test your connection, try `ping ubuntu@192.168.55.1`, or skip directly to using `ssh ubuntu@192.168.55.1`. You'll be prompted for the password (`ubuntu`).

Lab Overview

In this lab, you will learn to use ROS2 Humble to program your Rosbots. You will get oriented to connecting and developing on the bots and then use a builtin package to teleoperate the bot from your keyboard. Then, you will write your own ROS2 package to teleoperate the bots with a controller. By the end of this lab, you will be able to:

- Use ROS2 packages and topics
- Write a launch file
- Make your robot drive!
- Write your own ROS2 node to control your robot with a joystick.

Lab Procedure

1. Clone MEGN 441 git repository into a folder of your own for your team. Below, I've assumed that you've copied it directly into your home directory (~)
2. Get access to your robot through Nomachine or SSH as described in Connection Information above.
3. Copy your `ros2_ws/src` directory onto the robot. In Nomachine, this can be done from the `!M` logo in the top right. To copy using the command line, use the command below. `rsync` is a very useful copy command either from one device to another or just one folder to another. The `-u` flag in the command tells it to "update," which means it only copies over *new* files and saves a lot of time.

```
cd ~/megn441/ros2_ws
rsync -ruv src ubuntu@192.168.149.1:~/ros2_ws/
```

4. On the robot, build the ros2 workspace by navigating to ~/`ros2_ws` and using `colcon`. See course notes for support on this, and **don't forget to source `install/setup.bash`**.
5. If there is nothing currently running on the robot, run the launch file `bringup.launch.py` from the package `bringup`. This will run launch files within the `controller` package to get the drive functionality of the robot up and running.
6. Investigate some of the topics that are currently available with `ros2 topic list`. Use `ros2 topic info` to learn about the msg type for a few of the topics. If you use the `-v` flag, it will tell you more information. I'll have you report on 3 of the topics you learn about in the report.
7. To drive the robot, in a separate terminal, run the node (not launch file) `teleop_twist_keyboard` from the package `teleop_twist_keyboard`. Now you should be able to drive your robot around!
8. The next goal is to launch this driving all at once! First, we need to create a package, which we'll call `rosbot`. Navigate to `ros2_ws/src/` and use the following command (change `ament_python` to `ament_cmake` if you prefer C++). In the next step, we'll create a node called `teleop_joy`. The `--node-name` flag here will create that node and tell the package about its existence.

```
ros2 pkg create rosbot --build-type ament_python --node-name teleop_joy
    --dependencies teleop_twist_keyboard bringup
```

9. Create a directory within `~ros2_ws/src/rosbot` called `launch`. Copy a launch file template from the Lab1 folder here into `~/ros2_ws/src/rosbot/launch`. You will need to modify either `setup.py` or `CMakeLists.txt` to make `colcon` aware of the launch folder. See the ROS2 Humble launch docs for help. Use your launch file to call both `teleop_twist_keyboard` and `bringup.launch.py`. For `teleop_twist_keyboard`, you may want to launch it with `xterm`, so it pops up in its own window. Do this by adding the following within the Node to launch teleop_twist_keyboard:

```
prefix=['xterm -e'],
```

10. Then, write a node to read data from the `/ros_robot_controller/joy` topic and output to `/cmd_vel`. To figure out what the joystick does, make sure that `ros_robot_controller` is running, connect your controller, and use `ros2 topic echo ros_robot_controller/joy`. You'll be able to see the topic outputs when the controller buttons are pressed. See Lecture 4 materials to help you write the node. After you write your node and run it, you can drive your robot with the controller!

Lab Grading

The grading of each lab is based 50% on the successful completion of the lab. For this lab, that 50% breaks down into:

- 20% for successfully driving your robot around with the keyboard (upload a short video as described below)
- 20% for successfully writing a launch file
- 10% for successfully driving your robot around with the joystick (upload a short video as described below)

Lab Report Guidelines

The guidelines below will be used in grading your lab report. Be sure to include everything that the guidelines below mention for full credit!

1. Problem Statement

- Write a succinct 1-3 sentence description of the goals of this lab.

2. Methods

- Briefly describe the Rosbot, including a description of the drivetrain.
- Describe your contributions to the software of the robot for Lab 1.
- Explain 3 topics running on the Rosbots. What is the msg type that is sent to each of those topics? Which nodes publish to or subscribe to each of those topics?

3. Results

- Describe how driving with the keyboard works.
- Include a link to a video of piloting the bot with the keyboard.
- Report on how your controller driving node works.
- Include a link to a video of piloting the bot with the controller.
- Report on what is included in your launch file.
- Include a .zip file of the package you wrote with your submission and describe where in the folder yoru launch file and joystick node can be found.

4. Conclusions

- Discuss the performance of the different approaches to remotely piloting the robot.
- Discuss what your team's biggest lessons learned are from this lab.
- Discuss your current thoughts on pros vs cons of using ROS2 to create a robot.

AI Appendix

- Include a 1 paragraph reflection on the questions below, whether or not you used AI.
  - Did you use any AI to support support the completion of this lab? Why or why not?
  - If no, how could it have helped? What did you gain by avoiding AI use?
  - If yes, how was your experience of using it? How did it help? What did you miss out on by using AI?
  - If you used AI, what resources do you think it used in generating its answers?
  - If you used AI, please copy and paste your interactions below: